

Concepto 1

Core avanzado: threading, manejo de archivos y serialización.

De acuerdo a indicaciones del profesor, la primera parte de este concepto, threading o Hilos, solo se explicará conceptualmente, sin concretarse a código.

Thread, como concepto en computación, es un camino independiente donde se puede ejecutar algo. Con ejecutar ciertos comandos en Java, la JVM puede crear nuevos threads y ejecutarlos, y estos pueden trabajar de manera concurrente.

Runnable es algo que declaras para definir que trabajo se debe hacer en una thread, donde la thread puede aceptar como entrada un Runnable. Un runnable en así no se asigna ni crea una thread.

Para una revisión rápida de concurrencia, se pueden hacer ciclos simples de contador que impriman en pantalla, demostrando que con diferentes threads se pueden ejecutar tareas simultáneamente, no una tras la otra, si es que la computadora cuenta con diferentes cores en CPU. Si por alguna razón solo tienes single core, JVM puede simular concurrencia al pasar rápidamente entre ellos.

Executor service, cosa que llegó en Java 5, agrega una nueva herramienta, ya que andar creando threads individualmente no es tan eficiente. Se trata del executor service donde se maneja el concepto de thread pooling, un framework en donde un pool de threads se mantiene en reserva para ejecutar tareas, definida desde un inicio, es decir que puedes tener un pool de 2, 3 o más threads, tu executor toma ese pool, y dependiendo de la carga de trabajo se le pueden asignar tareas a todos los threads disponibles si es necesario.

Lo importante es el problema de concurrencia, que se refiere a la situación de que pasa si dos threads manejan el mismo dato, y si lo pones sin mitigar este problema, tu resultado cambia dependiendo de cómo se sincronizan los threads. Esto puede ser conocido como race condition.

Synchronized es una solución a esto, ya que define que solo un thread a la vez puede ejecutar esta sección de objeto o bloque de código. Otra manera más directamente relacionada con contadores es el uso de AtomicInteger, que es garantizado por Java de ser posible usarle de manera segura a través de varias threads.

El concepto de collections también presenta el mismo problema, ya que errores pueden ocurrir si varios threads intentan acceder como por ejemplo, una objeto lista de strings. Java nos ofrece colecciones concurrentes como ConcurrentHashMap entre otros, donde múltiples threads pueden manejarlos de manera segura.

También hay nuevas herramientas y conceptos que se le fueron agregando a Java a través del tiempo, como CompletableFuture, virtual threads, y demás para manejar ejecuciones asíncronas, pero esto son los conceptos básicos de threads en Java.

La lectura y escritura de archivos con IO, NIO y posiblemente NIO2 se basaran de acuerdo al proyecto provisto por el profesor, de su repositorio de curso Java 17 21, chapter 14.

Iniciando en el código: `..\chapter14\src\com\curso\streams\v1\Principal03` (Filepath Windows)

Vemos aplicado el uso de streams y el paquete de java.io. Se guarda un string con el valor del camino completo del user.dir, que en el caso de este código se asumió que es `../chapter14`, abrimos un objeto tipo file fileOrigen para leer un archivo origen.txt con la ubicación dada del string guardado mas `../data/` , junto con otro objeto tipo file fileDestino, esta vez apuntando a un archivo destino3.txt en la misma carpeta anterior.

Sin olvidar mandar el IOException para que nuestro código nos mande avisos si hay error en el propio proceso en su, hablamos a una función copyTextFile, mandando nuestros dos files, hacemos nuestra operación creando objetos de tipo bufferedreader y bufferedwriter, que hacen mas eficiente en memoria estos procesos de lectura y escritura al mandarlo a memoria en pedazos, o “buffers”, y mientras nuestro archivo no lea null del objeto reader, se va copiando al writer línea por línea.

Todo esto manejando el sistema IO de java, que esta desde classic Java, y el sistema try with resources, donde gracias al keyword try en java, estas operaciones desde java 7 pueden automáticamente cerrar resources, que pueden ser files, streams o demás.

Siguiente es `..\chapter14\src\com\curso\streams\v1\Principal04`, donde nuestro código es casi lo mismo que el anterior ejemplo, solo que sustituyendo bufferedwriter, con printwriter, que nos da acceso a ciertos métodos que fácilmente aceptan mas data types y los convierte a strings con print(), println() y más. Mientras que con bufferedwriter es mas limitado a caracteres y arreglos de estos, y strings, con métodos limitados a ellos.

De aquí pasamos a manejar protocolo NIO.2 de java con archivo `..\chapter14\src\com\curso\streams\v1\PrincipalPath01`, donde el código otra vez llamamos a `../chapter14/data/origen.txt`, para demostrar que importando paquetes para implementar Files, Path y Paths, podemos tener una lectura y copiado mas eficientes.

Grabamos otra vez la dirección del directorio, y esta vez creamos un objeto path, que aceptara como valor un objeto paths que es capaz de ubicar un archivo con el método get, este método recibiendo entrada de dirección de directorio guardado y strings “data” y “origen.txt”.

Ya con la dirección origen guardada y creando otros objetos path para los destinos, podemos llamar a tres tipos de métodos diferentes en Files para leer, que es

readString, readAllBytes, y readAllLines. Divididas en sus propias funciones, podemos copiar a destino con writeString, que acepta el readstring, o write, que acepta el readallbytes y readallLines. Con esto demostramos que java NIO.2 puede manejar lecturas por string, por byte o por línea.

Para terminar, en `..\chapter14\src\com\curso\streams\v1\PrincipalPath02`, podemos ver que un método de readLazily, que es mas eficiente en el uso de memoria al no guardar todo la entrada en memoria, y se ve que se lee el archivo sin problema, imprimiendo en pantalla. Incluso se puede manejar con operaciones de string, donde filtramos ciertas líneas del archivo de origen y estas ya no se imprimen.

Pasando ahora a serialización, que es el concepto de convertir objetos a datos transferibles y de vuelta en java. Vemos que en los archivos de código `..\chapter14\src\com\serializable\v0\Gorilla.java`, `PrincipalObjectInput.java` y `PrincipalObjectOutput.java`, creamos un objeto Gorilla con varios atributos, su toString, y sus getters y setters, y lo hacemos serializable, que hace posible aplicar este tema. En PrincipalObjectInput, asumiendo que tenemos un archivo gorilla.data que tenga el mismo orden que nuestra clase gorilla, observamos que en la función de readFromFile, del archivo gorilla.data pasa de un FileInputStream a bufferedInputStream, a ObjectInputStream, convirtiendo los datos para que se puedan guardar en un ObjectInputStream, y que cada objeto que sea una instancia de gorilla se guarde en un arreglo de gorillas. Al final el arreglo de gorillas se imprime en consola demostrando que la clase gorilla puede guardar datos del archivo gorilla.data.

La contraparte de esto es en PrincipalObjectOutput, en donde creamos una lista de gorilas en arreglo llamando a su constructor por objeto, y lo mandamos a guardar en una nueva función saveToFile, que con un for leemos los gorillas en la lista, y se serializa a ObjectOutputStream, de ahí a BufferedOutputStream y luego FileOutputStream, para crear un gorilla.data en directorio.

Estos codigos demuestran que la clase gorilla puede ser serializada para poder ser convertida a los streams apropiados para escribir y leer archivos, pasando de objeto a buffered stream a file stream. Una ultima cosa que notar, al hacer un objeto serializable java agrega un identificador del objeto default que puedes definir por tu cuenta, para mantener la validez de los datos, ya que, dependiendo de la versión de java u otro tipo de modificación posible para la clase, los bytes de la clase pueden cambiar. Al declarar el identificador de la clase y java lo lee, si estos son iguales puedes estar seguro que las clases son identicas.